

Réponse — Questions API app agent (besoins-api-app-agent.pdf, 05/08/2026)

Pour : dev app agent (Expo React Native) De : équipe backend NovAcces.Api Répond point par point au rapport du 05/08/2026. Deux corrections de code livrées en même temps que cette réponse (voir §Corrections livrées) : `POST /api/agent/shift/end` et l'annotation Swagger (sécurité + réponses typées) sur toute la surface agent.

P0 — Les 3 bloquants

Q1. Preuve de possession de la clé du device

Vos 5 hypothèses testaient chacune soit le bon encodage, soit le bon message, jamais les deux ensemble.

```
Message signé : "{ticket}|{deviceInstanceId}" (UTF-8, séparateur "|" littéral)
Algorithme : ECDSA P-256 / SHA-256 (ES256)
Encodage sig. : IEEE P1363 (r||s concaténés, 64 octets pour P-256) — PAS DER
Encodage texte : Base64URL (pas Base64 standard)
Format clé pub : PEM SPKI ("-----BEGIN PUBLIC KEY-----"), P-256 uniquement
Ticket : secret aléatoire opaque (PAS un JWT), 256 bits, remis en clair
une seule fois dans le QR d'enrôlement. Usage unique, durée
de vie 1-60 min (Enrollment:TicketLifetimeMinutes, défaut 60).
```

Extrait exact de la vérification serveur (`DeviceEnrollmentEndpoints.cs`) :

```
var signature = Base64UrlDecode(proofSignature);
var message = Encoding.UTF8.GetBytes($"{ticket}|{deviceInstanceId}");

using var ecdsa = ECDSA.Create();
ecdsa.ImportFromPem(devicePublicKeyPem);
return ecdsa.VerifyData(message, signature, HashAlgorithmName.SHA256);
```

`.NET ECDSA.SignData` produit nativement du P1363 — c'est probablement ça qui manquait côté mobile si votre lib signe en DER par défaut (cas fréquent des libs crypto JS/RN).

Q2. Authentification du terminal

Réponse de succès de `/api/device-enrollments/activate` :

```
{
  "terminalId": "guid",
  "label": "string",
  "siteIds": ["string"],
  "apiKey": "string", // secret OPAQUE, pas un JWT — pas de champ expiresAt
  "enrolledAt": "date"
}
```

`apiKey` est stable jusqu'à révocation manuelle par l'admin (pas de rotation automatique, pas d'endpoint de renouvellement).

Point critique, probablement votre second blocage une fois Q1 résolu : sur `/api/agent/*` et sur toutes les routes du contrat (`/api/site/config` , `/api/offline-list` , `/api/scan/sync` , ...), la policy

AgentTerminal exige la présence littérale de l'en-tête X-Api-Key sur **chaque requête, même si un Authorization: Bearer valide est déjà présent**. Le Bearer (jeton de poste ou jeton agent) ne remplace jamais la clé de terminal — les deux doivent être envoyés ensemble en permanence :

```
X-Api-Key: <clé remise à l'activation> <- toujours
Authorization: Bearer <jeton de poste> <- en plus, après shift/start
```

X-Site-Id : lu et revalidé contre la liste de sites autorisés du terminal **uniquement** quand l' Authorization commence par Bearer , sur le groupe de routes du contrat React Native. Pour un terminal **mono-site** authentifié seulement par X-Api-Key , le site est déduit du terminal lui-même et l'en-tête est ignoré (mais sans effet indésirable si vous l'envoyez quand même). Pour un terminal **multi-sites**, X-Site-Id est obligatoire (401/403 sinon) et revalidé côté serveur — jamais de site deviné par défaut.

GET /api/agent/sites existe, renvoie un tableau nu de chaînes (pas de DTO enrichi) : ["site-id-1", "site-id-2"] . Le site choisi se transmet ensuite via l'en-tête X-Site-Id sur chaque requête suivante — **aucun état serveur** (pas de session collante), à répéter à chaque appel.

Q3. Structure du QR visiteur

Ce n'est **pas** un JWS RFC 7515 standard (pas de sérialisation compacte header.payload.signature , pas de champ alg dans un header). C'est une enveloppe JSON custom :

```
{
  "PayloadB64Url": "...", // JSON UTF-8 { VisitId, VisitToken, Exp } en Base64URL
  "SignatureB64Url": "...", // signature ES256 (P1363, Base64URL) des octets du payload JSON
  "KeyId": "current" // "kid" – en clair, HORS signature
}
```

Claims du payload (une fois décodé) : VisitId (Guid), VisitToken (Guid), Exp (Unix seconds). **Rien d'autre** — pas de siteId , pas de nom (RGPD).

KeyId correspond au kid de GET /api/keys/public ("current" , ou l'une des entrées de RetiredKeys pendant une rotation). Un keyId absent ou inconnu fait juste échouer la recherche de clé de vérification — safe par construction, pas besoin de le valider vous-même en amont.

Important

Q4. Énumération exhaustive de verdictCode

Maintenant documentée dans Swagger (description de POST /api/scan), et ci-dessous pour référence immédiate :

GRANTED – accès accordé
CHECKED_OUT – sortie enregistrée (jamais bloquée, même QR révoqué)
INVALID_SIGNATURE – QR introuvable / signature invalide / cryptographiquement expiré
INVALID_CODE – code de secours introuvable (POST /api/scan/manual-code uniquement)
DENIED_Excluded – visiteur sur liste d'exclusion
DENIED_NoActiveEntry – scan sortie sans entrée active
DENIED_SuspectedDuplicate – déjà sur site, re-scan entrée (suspicion de copie)
DENIED_Revoked – QR révoqué
DENIED_CycleAlreadyClosed – cycle entrée/sortie déjà bouclé
DENIED_AlreadyConsumed – QR à passage unique déjà consommé
DENIED_TooEarly – avant la fenêtre (-20 min)
DENIED_TooLate – après la fenêtre (+15 min), ou hors 30 jours
DENIED_NonBusinessDay – mode 30 jours présenté un jour non ouvré

Traitez tout code hors cette liste comme un refus (jamais comme une autorisation) — c'est une exigence de sûreté, pas une précaution excessive.

Q5. Format d'erreur et sémantique 409/429

Pas de `ProblemDetails` sur les refus métier : { "error": "message en français" } , parfois { "error": "...", "details": [...] } . Seules les exceptions non gérées (500) suivent RFC 7807.

- **409 Conflict** : conflit métier (doublet de visite active, écarts de resynchronisation POST /api/scan/sync).
- **410 Gone** : ticket d'enrôlement invalide/expiré/déjà utilisé (POST /api/device-enrollments/activate uniquement).
- **429 Too Many Requests** : rate limiting — 30 scans/min par IP+terminal sur les routes `sensitive` , pas de corps JSON custom (généré par le middleware .NET lui-même).

Q6. direction

Déjà un enum côté serveur (`CheckpointDirection.Entry` / `.Exit`), parsé insensible à la casse depuis le string du DTO. Toute autre valeur -> 400. Rien à changer côté contrat, c'est déjà fiable.

Q7. checkpointId

String libre, **jamais validée** contre GET /api/site/config — transportée telle quelle jusqu'au journal (max 100 caractères), purement informative pour l'audit. Envoyez ce que vous voulez d'identifiant lisible.

P1 — DTOs de réponse

```
// POST /api/agent/shift/start
{ matricule: string, displayName: string, shiftToken: string, expiresAt: string }

// GET /api/agent/sites
string[]

// GET /api/site/config
{
  siteLabel: string,
  postes: { id: string, nom: string }[],
  params: { fenetreAvantMin: number, fenetreApresMin: number, ttlListeLocaleHeures: number }
}

// GET /api/agent/expected-today
{ visitorName: string, status: "attendu"|"sur site"|"sorti"|"révoqué"|"non venu",
  windowStart: string|null, windowEnd: string|null }[]

// GET /api/offline-list (contrat – voir Q9 pour la forme de signedList)
{ generatedAtUtc: string, expiresAtUtc: string,
  visits: { visitId, nom, mode, fenetreDebut, fenetreFin, statut, present }[],
  signedList: string }

// POST /api/scan et POST /api/scan/manual-code
{ isGranted: boolean, isCheckOut: boolean, isSecurityEvent: boolean,
  verdictCode: string, visitorName: string|null, overstayMinutes: number|null,
  presenceMinutes: number|null }

// POST /api/scan/sync
{ accepted: number, conflicts: { visitId: string, raison: string }[] }
// 200 si conflicts vide, 409 sinon (même corps dans les deux cas)
```

Ces types sont désormais dans le contrat OpenAPI lui-même (`GET /swagger/v1/swagger.json`) — plus besoin de les déduire par observation.

À trancher

Q8. `/api/agent/resync` vs `/api/scan/sync`

`/api/scan/sync` **est la route destinée à React Native** — c'est déjà écrit dans le code (`AgentContractEndpoints.cs` , désormais aussi dans la description Swagger de l'opération).

`/api/agent/resync` reste actif pour l'app MAUI existante (autre client, historique) ; ne l'utilisez pas. Les deux sont testés et fonctionnels, mais les corps de requête divergent volontairement — n'essayez pas de faire un seul client générique pour les deux.

Q9. `/api/agent/offline-list` vs `/api/offline-list`

`/api/offline-list` (**sans `/agent`**) **est la route contrat, pour vous.**

`signedList` est une **chaîne JSON**, pas un JWS compact brut :

`"{"PayloadB64Url":"...","SignatureB64Url":"...","KeyId":"current"}"` — il faut `JSON.parse` deux fois : une fois la réponse HTTP, une fois le contenu de `signedList` . Structure du payload une fois décodé : liste d'entrées `{ visitId, visitToken, scheduledAt, isExcluded, isOnSite }` (plus enrichi côté `visits[]` en clair : nom, mode, statut — voir P1 ci-dessus). Fréquence de rafraîchissement recommandée : avant expiration du TTL (`ttlListeLocaleHeures` dans `GET /api/site/config` , borné à 4h max).

Ce qui manquait

Q10. Code de secours

Généré à la **création de la visite** (`POST /api/visits`), champ `manualCode` de la réponse — remis en clair **une seule fois** (jamais récupérable ensuite, seule son empreinte est stockée). Format : 8 caractères `XXXX-XXXX` , alphabet sans caractères ambigus (`ABCDEFGHIJKLMNOPQRSTUVWXYZ23456789`). Pas de TTL dédié : valide tant que les mêmes règles de fenêtre que le QR s'appliquent (-20/+15 min, ou 30 jours ouverts). **Non utilisable hors ligne** — sa résolution nécessite une recherche en base, contrairement au QR vérifiable cryptographiquement sans réseau ; affichez clairement cette limite à l'agent plutôt que d'échouer en silence. Réponse : le même `ScanResponseDto` que `POST /api/scan` .

Q11. Fin de poste — livré avec cette réponse

```
POST /api/agent/shift/end
Headers: X-API-Key, X-Shift-Token
Body: (vide)
-> 200 OK
```

Idempotent : rejouer l'appel, ou l'appeler après qu'un autre agent a démarré un nouveau poste sur le même terminal, ne fait rien et renvoie 200 dans tous les cas — jamais d'erreur. Effet : le poste clos n'attribue plus les scans au matricule parti (repli sur l'identité du terminal), même si votre app oublie de purger le jeton local avant son expiration naturelle. À appeler systématiquement à la déconnexion / au changement d'agent.

Q12. Hub SignalR

Existe et est câblé : `/hubs/scan-events` (policy incluant le rôle Agent — distinct de `/hubs/scan` , réservé au dashboard sûreté). Authentification par jeton en **query string** `?access_token=<jwt>` (SignalR/WebSocket ne portant pas facilement un en-tête `Authorization: custom`) — le token peut être le Bearer agent ou le jeton de poste. Site transmis en query string `?site=...` ou en-tête `X-Site-Id` à la connexion, revalidé côté serveur. Messages : `VisitRevoked` , `VisitCreated` .

Corrections livrées avec cette réponse

- `POST /api/agent/shift/end` (Q11) — nouvel endpoint, testé (idempotence + non-réattribution après clôture).
- Swagger** — `components.securitySchemes` déclare désormais `Bearer` et `ApiKey` ; chaque opération authentifiée déclare le schéma réellement exigé (les deux ensemble pour `AgentTerminal`) ; les réponses de toute la surface agent (`/api/device-enrollments/activate` , `/api/agent/*` , `/api/scan*` , `/api/health` , `/api/keys/public` , `/api/site/config` , `/api/offline-list` , `/api/scan/sync`) sont typées avec les DTOs réels. Vous pouvez régénérer votre client à partir de `GET /swagger/v1/swagger.json` .

Récapitulatif

#	Statut
Q1 Message signé pour <code>proofSignature</code>	[OK] Répondu ci-dessus

#	Statut
Q2 Jeton Bearer + X-API-Key obligatoire ensemble	[OK] Répondu ci-dessus
Q3 Claims + structure du QR JWS	[OK] Répondu ci-dessus
Q4 Liste exhaustive des verdictCode	[OK] Répondu ci-dessus + Swagger
Q5-Q7 Erreurs, direction , checkpointId	[OK] Répondu ci-dessus
P1 DTOs de réponse	[OK] Répondu ci-dessus + Swagger
Q8 /api/scan/sync (pas /api/agent/resync)	[OK] Tranché
Q9 /api/offline-list (pas /api/agent/offline-list), JWS enveloppé en string	[OK] Tranché
Q10 Code de secours	[OK] Répondu ci-dessus
Q11 POST /api/agent/shift/end	[OK] Livré
Q12 Hub SignalR /hubs/scan-events	[OK] Confirmé, existe